

积分问题影响范围及修复方案

一、问题结论

当前积分问题不是单点的“9 月 30 日积分过期”问题，而是用户积分余额与积分批次账长期不一致。

系统目前存在两套积分数据：

- `eb_user.integral`：用户当前可用积分余额
- `eb_integral_record.remaining_integral`：每批积分剩余可用积分，用于判断消费顺序和过期

正常情况下应满足：

用户当前积分 = 所有已生效批次剩余积分之和

但当前查询发现大量用户不一致，约 319482 个用户存在差异，且多数是批次账大于用户实际余额。

因此，不能直接执行现有积分过期任务，否则可能造成重复扣减、错误过期或用户余额异常。

二、影响范围

当前问题会影响以下功能：

- 积分过期金额计算
- 即将过期积分展示
- 积分消费批次扣减
- 积分明细展示
- 用户积分余额与积分流水对账
- 后续积分过期任务

主要风险包括：

- 用户积分被重复扣减

- 用户积分被扣成负数
- 用户余额不足但仍生成积分过期明细
- 已消费过的积分被再次过期
- `eb_integral_record.remaining_integral` 持续虚高
- 9月30日过期金额大于用户实际可用积分

三、根因分析

1. 统一扣积分方法跨批次扣减不完整

正常购物抵扣、积分商品兑换、优惠券兑换会调用：

```
IntegralRecordServiceImpl.subtractIntegral(...)
```

该方法会扣减 `eb_user.integral`，但在扣减积分批次账时，只更新了最后一个被部分扣减的批次。

如果一笔消费跨多个积分批次，前面被完整消耗的批次没有更新为 0。

示例：

```
A 批 remaining_integral = 100  
B 批 remaining_integral = 50
```

用户消费 120

正确结果应为：

```
A 批 remaining_integral = 0  
B 批 remaining_integral = 30
```

当前代码实际可能变成：

```
A 批 remaining_integral = 100  
B 批 remaining_integral = 30
```

用户余额已经扣了 120，但批次账只扣了 20，导致批次账虚高。

2. 抽奖消耗积分绕过统一扣减逻辑

抽奖消耗积分时直接调用：

```
userEntityMapper.updateIntegralSub(...)
```

该操作只扣减 `eb_user.integral`，没有同步扣减 `eb_integral_record.remaining_integral`。

因此抽奖消耗会导致：

用户余额减少
积分批次账不减少

3. 抽奖消耗错误新增积分批次

抽奖消耗积分后，代码又新增了一条 `integral_status = 1` 的积分记录。

抽奖消耗属于积分扣减，不应该新增可用积分批次。

这会进一步放大 `eb_integral_record.remaining_integral`，制造虚假的可用积分批次。

4. 抽奖中奖获得积分批次金额写错

抽奖中奖获得积分时，当前代码将：

```
remaining_integral = 用户原积分 + 本次获得积分
```

正确逻辑应为：

```
remaining_integral = 本次获得积分
```

该问题会导致中奖积分批次金额被放大，尤其是高积分用户中奖时，会产生非常大的批次账差异。

5. 过期逻辑本身存在错误

当前过期任务存在以下问题：

- 查询 `integral_status = 0`，但 `0` 是待生效，真正可过期的是 `1` 已生效
- 使用 `create_time <= 当前时间 - 1 年` 判断过期，应该使用 `expire_time`
- 使用 `SUM(integral)` 汇总过期金额，应该使用 `SUM(remaining_integral)`
- 用户余额扣减采用先查询再 `set` 的方式，存在并发覆盖风险

四、修复方案

修复分三步处理。

第一步：修复代码，阻止问题继续扩大

1. 修复 `subtractIntegral()` 跨批次扣减逻辑

每一批被消费的积分都必须更新 `remaining_integral`。

修复核心逻辑：

```
if (remainingIntegral >= totalRemineIntegral) {  
    // 当前批次部分扣减  
    updateIntegralRecord(recordId, totalRemineIntegral);  
    break;  
} else {  
    // 当前批次全部扣完，也必须更新  
    updateIntegralRecord(recordId, remainingIntegral);  
    totalRemineIntegral -= remainingIntegral;  
}
```

2. 修复抽奖消耗积分逻辑

抽奖消耗积分不能直接调用：

```
userEntityMapper.updateIntegralSub(...)
```

应改为统一调用：

```
integralRecordService.subtractIntegral(  
    user.getUid(),  
    lottery.getIntegral(),  
    IntegralSource.LOTTERY_CONSUME,  
    "抽奖活动消耗(" + lottery.getMarketName() + ")",  
    true,  
    lottery.getMarketId(),  
    orderSn  
);
```

同时删除抽奖消耗后新增 `IntegralRecord` 的错误逻辑。

3. 修复抽奖中奖获得积分逻辑

将：

```
integralRecord.setRemainingIntegral(user.getIntegral() + lotteryAward.
```



改为：

```
integralRecord.setRemainingIntegral(lotteryAward.getIntegral());
```

4. 修复积分过期逻辑

过期查询条件应改为：

```
integral_status = 1  
AND remaining_integral > 0  
AND expire_time <= #{executeTime}
```

过期金额应改为：

```
SUM(remaining_integral)
```

用户积分扣减应使用原子扣减：

```
UPDATE eb_user
SET integral = integral - #{expireIntegral},
    integral_dec = integral_dec + #{expireIntegral}
WHERE uid = #{uid}
AND integral >= #{expireIntegral};
```

第二步：修复历史数据，校准积分批次账

以 `eb_user.integral` 作为当前真实可用积分主账。

修复目标：

每个用户的已生效批次剩余积分合计 = `eb_user.integral`

修复原则：

- 不直接修改用户余额
- 只调整 `eb_integral_record.remaining_integral`
- 对于 `active_record_integral > user_integral` 的用户，按
 `expire_time ASC, integral_record_id ASC` 从最早过期批次开始清理多
 余积分
- 清理后重新核对账务一致性
- 修复前必须备份相关表

修复后校验 SQL：

```
SELECT
    u.uid,
    u.integral AS user_integral,
    IFNULL(SUM(r.remaining_integral), 0) AS active_record_integral,
    IFNULL(SUM(r.remaining_integral), 0) - u.integral AS diff
FROM eb_user u
LEFT JOIN eb_integral_record r
    ON r.uid = u.uid
    AND r.integral_status = 1
    AND r.remaining_integral > 0
GROUP BY u.uid, u.integral
HAVING diff <> 0
ORDER BY diff DESC;
```

目标是差异清零，或降到可解释范围。

第三步：处理 9 月 30 日过期积分

代码修复和历史批次账校准完成后，再执行 9 月 30 日过期。

查询待过期积分：

```
SELECT
    uid,
    SUM(remaining_integral) AS expire_integral
FROM eb_integral_record
WHERE integral_status = 1
    AND remaining_integral > 0
    AND expire_time <= '2026-09-30 23:59:59'
GROUP BY uid;
```

执行过期时：

- 标记对应积分批次为已过期
- 扣减用户实际余额
- 生成 `integral_source = 7` 的积分过期明细
- 明细金额使用实际过期的 `remaining_integral`

五、执行顺序

1. 备份生产表：

```
CREATE TABLE bak_eb_user_20260930 AS SELECT * FROM eb_user;
CREATE TABLE bak_eb_integral_record_20260930 AS SELECT * FROM eb_integ
CREATE TABLE bak_eb_integral_details_20260930 AS SELECT * FROM eb_inte
```



2. 修复代码并上线。
3. 在测试库验证历史数据修复脚本。
4. 执行历史批次账校准。
5. 校验用户余额与批次账一致性。
6. 执行 9 月 30 日积分过期。

7. 再次校验用户余额、积分批次、积分明细三方一致性。

六、汇报口径

本次问题根因是积分扣减没有完整同步积分批次账，叠加抽奖模块错误写入积分批次，导致历史批次账大面积虚高。

因此不能直接执行 9 月 30 日过期任务。正确处理方式是：

先修代码防止新增异常
再按用户余额校准历史批次账
最后执行积分过期

当前用户可用积分以 `eb_user.integral` 为准，历史积分批次账按最早过期优先原则进行校准。